Prueba Técnica

Desarrollador Full Stack — .NET 10 + Angular (v20+)

Proyecto de evaluación: Gestor de Inventario

1. Introducción

Este documento define el proyecto que debe desarrollar el candidato como parte de la prueba técnica. El proyecto debe construirse siguiendo estrictamente el documento "Estándar de Arquitectura (.NET + Angular)" entregado junto con esta prueba: Clean Architecture + CQRS en el backend, y el Estándar de Desarrollo Frontend Angular del equipo AGAVAL en el frontend.

Adicionalmente, se incluye un cuestionario técnico (sección 4) que debe resolverse junto con la entrega del proyecto.

2. Base de Datos — SQL Server

El candidato debe crear la base de datos y las tablas manualmente en SQL Server (o mediante migraciones de EF Core equivalentes a este script), respetando los nombres y tipos indicados a continuación. La tabla Categorias se entrega con datos precargados (seed); no es necesario construir su gestión (ver sección 3.5, opcional).

```
CREATE DATABASE GestorInventarioDB;
GO
USE GestorInventarioDB;
GO

CREATE TABLE Categorias (
    Id            INT IDENTITY(1,1)   NOT NULL,
    Nombre        NVARCHAR(100)        NOT NULL,
    Activo        BIT                  NOT NULL DEFAULT (1),
    CONSTRAINT PK_Categorias PRIMARY KEY (Id)
);
GO

CREATE TABLE Productos (
    Id            INT IDENTITY(1,1)   NOT NULL,
    Nombre        NVARCHAR(150)        NOT NULL,
    Descripcion   NVARCHAR(500)        NULL,
    Precio        DECIMAL(10,2)        NOT NULL,
    Stock         INT                  NOT NULL DEFAULT (0),
    StockMinimo   INT                  NOT NULL DEFAULT (5),
    CategoriaId   INT                  NOT NULL,
    FechaCreacion DATETIME2            NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT PK_Productos PRIMARY KEY (Id),
    CONSTRAINT FK_Productos_Categorias FOREIGN KEY (CategoriaId)
        REFERENCES Categorias (Id),
```

```
    CONSTRAINT CK_Productos_Stock CHECK (Stock >= 0)
);
GO

-- Datos iniciales (seed)
INSERT INTO Categorias (Nombre) VALUES
    (N'Electrónica'), (N'Oficina'), (N'Aseo');
GO
```

La tabla MovimientosInventario (registro de entradas/salidas de stock) es opcional, ver sección 3.5:

```
-- OPCIONAL
CREATE TABLE MovimientosInventario (
    Id            INT IDENTITY(1,1)   NOT NULL,
    ProductoId    INT                  NOT NULL,
    TipoMovimiento VARCHAR(10)         NOT NULL, -- 'ENTRADA' / 'SALIDA'
    Cantidad      INT                  NOT NULL,
    Fecha         DATETIME2            NOT NULL DEFAULT (GETDATE()),
    Observacion   NVARCHAR(255)        NULL,
    CONSTRAINT PK_MovimientosInventario PRIMARY KEY (Id),
    CONSTRAINT FK_Movimientos_Productos FOREIGN KEY (ProductoId)
        REFERENCES Productos (Id),
    CONSTRAINT CK_Movimientos_Tipo CHECK (TipoMovimiento IN ('ENTRADA','SALIDA'))
);
```

3. Proyecto a Desarrollar: Gestor de Inventario

3.1. Descripción general

Construir una aplicación full stack para la gestión de productos de una pequeña empresa: registro de productos, asociación a una categoría existente y visibilidad sobre productos con stock bajo, aplicando estrictamente la arquitectura descrita en el documento de estándar.

3.2. Modelo de datos

Entidad	Campo	Tipo / Regla
Categoria	Id, Nombre, Activo	Precargada por seed; se usa como lista para asociar productos
Producto	Id, Nombre, Descripcion, Precio, Stock, StockMinimo, CategoriaId	Precio > 0, Stock >= 0
Producto	Stock bajo	Se considera "stock bajo" cuando Stock < StockMinimo

3.3. Funcionalidades requeridas — Backend (.NET)

- CRUD completo de Producto (Commands y Queries separados, según CQRS): crear, listar, obtener por id, editar y eliminar.
- Al crear/editar un producto, seleccionar una CategoriaId existente (de las precargadas por seed).
- Endpoint para listar productos con stock bajo (GET /api/productos/stock-bajo).

- Endpoint para ajustar stock de un producto (entrada/salida).
- Validaciones de negocio (Precio > 0, Stock >= 0, Nombre obligatorio).
- Seed de datos iniciales (categorías y algunos productos de ejemplo).
- Documentación de la API con Swagger.

3.4. Funcionalidades requeridas — Frontend (Angular)

- Feature "productos": listado en tabla (Angular Material) con indicador visual para stock bajo.
- Formulario de creación / edición de producto, con formularios reactivos tipados (la categoría se selecciona de una lista).
- Filtro de productos por categoría y por estado de stock (bajo / normal).
- Manejo de estado con Signals, componentes standalone con OnPush.
- Rutas de la feature cargadas con lazy loading.

3.5. Opcional / No requerido

Lo siguiente no es obligatorio para la entrega. Se considera un plus si el candidato decide implementarlo, pero no afecta negativamente si se omite:

- CRUD completo de Categoría (crear, editar, eliminar categorías) — tanto en backend como en frontend.
- Registro de movimientos de inventario (tabla MovimientosInventario).
- Autenticación y autorización.
- Paginación avanzada del listado de productos.
- Pruebas unitarias y/o de integración.
- Despliegue en la nube.

4. Cuestionario Técnico

Preguntas de respuesta abierta a resolver junto con el desarrollo del proyecto.

4.1. Preguntas Conceptuales

Backend

1. ¿Por qué ProductosController no debe acceder directamente al PersistenceContext de EF Core, sino pasar siempre por los Handlers de Application? ¿Qué se gana con esa decisión frente a la alternativa, más rápida de escribir, de usar el DbContext directo en el controlador?

2. Ante el requisito "al crear un producto con stock menor a su stock mínimo, se debe notificar por correo al área de compras", ¿por qué no debería implementar esa lógica directamente dentro de CrearProductoCommandHandler? ¿Qué alternativa de diseño elegiría y qué principio respalda esa decisión

Frontend

1. ¿Por qué el proyecto se construye 100% con componentes standalone en lugar de usar NgModules, como se hacía tradicionalmente en Angular? ¿Qué ventaja concreta le da esta decisión al lazy loading de features?

2. ¿Por qué se eligió ChangeDetectionStrategyOnPush en lugar de la estrategia por defecto para los componentes del proyecto? ¿Qué implicaciones tiene esa decisión sobre la forma en que se deben actualizar los datos en el componente?

CI/CD, Docker y Kubernetes

1. ¿Por qué definir los recursos de Kubernetes (Deployment, Service, ConfigMap, etc.) en archivos YAML versionados en el repositorio en lugar de crearlos manualmente con kubectl create desde la línea de comandos?
2. ¿Por qué empaquetar la aplicación en una imagen Docker en lugar de desplegar directamente el artefacto compilado sobre un servidor o VM? ¿Qué problema resuelve concretamente containerizar la app?

3. Por qué separar el pipeline en stages independientes de Build y Release/Deploy en lugar de tener un único job que compile y despliegue todo de una vez? ¿Qué riesgo mitiga esa separación?

5. Entregables y Criterios de Evaluación

Entregables

- Repositorio Git (backend y frontend).
- Script o migración de base de datos (según sección 2).
- README con instrucciones para ejecutar backend y frontend localmente.
- Respuestas al cuestionario conceptual (documento aparte o incluidas en el README).

Criterios de evaluación

Criterio	Qué se evalúa
Arquitectura backend	Separación correcta de capas, uso de CQRS, cumplimiento de la regla de dependencias.
Arquitectura frontend	Estructura core/shared/features, Signals, OnPush, standalone, lazy loading.
Cumplimiento del estándar	Ausencia de anti-patrones prohibidos (any, NgModules, !important, ::ng-deep, etc.).
Funcionalidad	El CRUD de Producto cumple con lo especificado en la sección 3 y corre sin errores.
Calidad de código	Nombrado, legibilidad, responsabilidad única, manejo de errores razonable.
Respuestas conceptuales	Comprensión demostrada de los principios de arquitectura, no solo la sintaxis.